

LEVEL 24

Chapter 24: Resource Library

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026



Chapter 24: Resource Library

← [Chapter 23: Performance Engineering](#) · [Index](#) · Next → [Chapter 25: Study Plans](#)

Curated deliberately small. Every resource here earns its place — this is not a "here are 40 links, good luck" dump. Where a channel/creator has multiple plausible URLs floating around (redirects, old handles), the current, verified handle is given. If a link ever 404s, search the channel/book name directly rather than assuming the resource is gone — creators do occasionally change handles.

24.A English YouTube Channels

Channel	What to watch	Difficulty	Why it's here	Order
ByteByteGo	The channel itself, browse by topic — created by Alex Xu (author of the <i>System Design Interview</i> books)	Beginner→Intermediate	The most consistently recommended starting point across every credible source checked for this roadmap — short (5–15 min), animation-driven, no fluff. Best for building the <i>visual</i> mental model of a concept fast.	1st — watch alongside reading Chapters 1–13 of this roadmap, topic-matched
Gaurav Sen	His dedicated System Design Playlist	Beginner→Intermediate	Longer-form than ByteByteGo, walks through reasoning step by step rather than just showing the end diagram — good for the "how would I have derived this" muscle this roadmap emphasizes. Widely reported as the most Hindi-speaker-friendly major English channel (clear pacing, no assumed jargon).	2nd , same phase as ByteByteGo

Channel	What to watch	Difficulty	Why it's here	Order
Hussein Nasser	His backend engineering and database-internals videos specifically (Postgres, Redis, networking protocols, gRPC)	Intermediate→Advanced	Genuinely different from the other two — live terminal demos, real protocol-level depth (TCP, database internals) rather than interview-framed content. The best channel here for Chapters 1, 5, and 7's deeper mechanics.	3rd , once fundamentals are solid — pairs naturally with your own hands-on Kafka/Redis/Postgres experience
Exponent	Their system design mock interview and company-specific videos	Intermediate→Advanced	Genuine mock-interview footage and company-specific breakdowns (including some Amazon/Meta-style content) — useful specifically for Chapter 28 (mock interviews) and Chapter 14 (communication), not for core concept-learning.	Use during mock interview practice phase , not week 1
freeCodeCamp	Search their channel for full-length "System Design Course" uploads (they periodically publish long-form full courses from various instructors)	Beginner	Good for a single long, free, structured sitting if you prefer one long video over a scattered playlist — treat as an alternative on-ramp, not required alongside ByteByteGo/Gaurav Sen.	Optional , alternative to 1st/2nd if you prefer long-form
AWS (Amazon Web Services)	"This is My Architecture" series and AWS re:Invent architecture talks specifically	Intermediate→Advanced	Real, production reference architectures from actual AWS customers — directly supports Chapter 12. More valuable <i>after</i> you know the	Pair with Chapter 12 specifically

Channel	What to watch	Difficulty	Why it's here	Order
			generic patterns, as a "here's how a real company actually wired this together" gut-check.	
Google Cloud Tech	Search for their system design / architecture-pattern content	Intermediate	Useful for cross-checking that your mental models aren't AWS-only — several patterns (pub/sub, global load balancing) are explained cleanly here even if you're AWS-focused day to day.	Optional, cross-reference only

A note on "Jordan has no life" and similar niche channels: not included — this research pass didn't find consistent, current, corroborated evidence they meet the bar this roadmap sets (complete, accurate, still-active playlists) versus the channels above. If you come across a channel not listed here, apply the same test before trusting it: is it a complete playlist (not a scattered handful of videos), is it recently active, and does it explain *reasoning*, not just show a finished diagram.

24.B Hindi / Hinglish YouTube Resources

Channel	What to watch	Difficulty	Why it's here	Order
Gaurav Sen	Same System Design Playlist as above	Beginner→Intermediate	Delivered in clear English but consistently reported across multiple sources as the most accessible major creator for Hindi-speaking engineers specifically — pacing and explanation style translate well even though it's not Hinglish-language content. Listed here again deliberately, since it's the single most corroborated recommendation for your specific situation across this research.	Same as English track

Channel	What to watch	Difficulty	Why it's here	Order
Chai aur Code (Hitesh Choudhary)	Search the channel directly for "system design" — the channel runs periodic dedicated system design cohorts/series (content and exact playlist links shift over time as new cohorts are run)	Beginner→Intermediate	A genuinely Hindi/Hinglish-native teaching channel (not just an English channel with Hindi-speaker-friendly pacing) with real architecture explanation, not just interview-answer scripts — matches the roadmap's explicit requirement to prefer instructors who teach real architecture. Check the channel's current uploads/ community tab for the latest live cohort, since Hitesh runs these periodically rather than as one static evergreen playlist.	Use alongside Gaurav Sen, not instead of

Honest gap to flag: the Hindi/Hinglish system-design content landscape is thinner and changes faster than the English one — fewer channels maintain a single complete, evergreen playlist the way ByteByteGo does. The two above were the most consistently corroborated across this research pass. If you find a promising Hindi-language channel not listed here, apply the same test: complete (not scattered) coverage, recent activity, and real architectural reasoning rather than "here's the answer, memorize it."

24.C Blogs

Blog	Best for	Link
System Design Primer	The single best free, structured written companion to this roadmap — 85,000+ GitHub stars, actively maintained, covers CAP, caching, load balancing, and includes real case studies (Twitter, Uber)	github.com/donnemartin/system-design-primer
ByteByteGo Newsletter/ Blog	Weekly, bite-sized, visual breakdowns of one system/concept at a time — good for spaced revision (Chapter 25) once you've done the main study pass	blog.bytebytego.com
High Scalability	The original "how does X actually work at scale" blog — deep, older but still foundational case studies	highscalability.com
	Official reference architectures for exactly the AWS-service decisions in Chapter 12	aws.amazon.com/architecture

Blog	Best for	Link
AWS Architecture Center		
Netflix Tech Blog	CDN/streaming architecture (Chapter 18, Problem 20), chaos engineering, microservices resilience	netflixtechblog.com
Uber Engineering Blog	Geospatial indexing, dispatch systems, Kafka-at-scale — directly supports Chapter 18, Problems 17/23	uber.com/blog/engineering
Airbnb Tech Blog	Marketplace/booking-system architecture, search ranking	airbnb.tech
Stripe Engineering Blog	The best real-world source for Chapter 20's payment/idempotency/reconciliation patterns from a company whose entire business is this problem	stripe.com/blog/engineering
Cloudflare Blog	CDN internals, DDoS mitigation, edge computing — supports Chapter 4.2 and Chapter 10	blog.cloudflare.com
LinkedIn Engineering	Feed/social-graph systems, Kafka's own origin story (Kafka was originally built at LinkedIn)	engineering.linkedin.com
DoorDash Engineering	Marketplace/dispatch systems directly analogous to Chapter 18, Problem 24's batching/fleet content	doordash.engineering
Martin Kleppmann's site	The author of <i>Designing Data-Intensive Applications</i> — talks, papers, and distilled versions of the book's hardest chapters	martin.kleppmann.com

(Meta/Google engineering blog URLs shift periodically as they reorganize their engineering site structure — search "[Meta/Google] engineering blog" directly closer to when you need it rather than relying on a URL fixed at the time this roadmap was written.)

24.D Books — In the Order to Actually Read Them

Must Read

1. System Design Interview - An Insider's Guide, Volume 1 (Alex Xu) — read the **entire book**; at 16 focused, practical chapters it's short enough that selective skipping isn't worth the risk of a gap, and it's the closest thing to a direct written companion to Chapters 1–3 and 16 of this roadmap. Start with Ch.1–3 (scale from zero, back-of-envelope estimation, the interview framework) before anything else in the book — they set up everything after.

2. Designing Data-Intensive Applications (Martin Kleppmann) — this is the deepest, most respected book in the field, but it's genuinely long and not every chapter is equally interview-critical. Read in this priority order:

- **Essential (read fully):** Ch.1 (Reliable, Scalable, Maintainable Applications — the conceptual foundation for this entire roadmap's Chapter 2), Ch.5 (Replication), Ch.6 (Partitioning), Ch.7 (Transactions), Ch.8 (The Trouble with Distributed Systems), Ch.9 (Consistency and Consensus) — these five map almost directly onto this roadmap's Chapters 5–6.
- **Worth reading, can skim the deepest internals:** Ch.3 (Storage and Retrieval — the B-Tree/LSM-tree comparison is genuinely useful, but you don't need to memorize every implementation detail), Ch.2 (Data Models — useful context for the SQL/NoSQL decision framework in Chapter 4.6).
- **Optional / lower interview priority, but valuable for genuine mastery:** Ch.4 (Encoding), Ch.10–12 (Batch Processing, Stream Processing, The Future of Data Systems) — excellent if you want depth beyond interview-readiness, not essential for it.

Optional (Strongly Recommended If You're Already Reading Volume 1)

3. System Design Interview, Volume 2 (Alex Xu & Sahn Lam) — 13 chapters, each one a fully worked advanced problem. Given your specific fintech/marketplace target companies (Chapter 19), read selectively rather than cover to cover:

- **High priority for you specifically:** Ch.11 (Payment System) and Ch.12 (Digital Wallet) — directly overlapping with this roadmap's Chapter 20 and highest-relevance for Razorpay/PhonePe/CRED-style interviews.
- **High priority given your geospatial/marketplace target companies:** Ch.1 (Proximity Service) and Ch.3 (Google Maps) — directly relevant to Careem/Uber/Talabat-style interviews.
- **Good general depth:** Ch.4 (Distributed Message Queue — reinforces Chapter 7 here), Ch.9 (S3-like Object Storage).
- **Lower priority for your specific goals, skip if time-constrained:** Ch.13 (Stock Exchange) — a fascinating but narrow, low-latency-trading-specific problem unlikely to appear in your target company list.

Advanced (Only If You Want to Go Beyond Interview-Readiness Into Genuine Depth)

4. Database Internals (Alex Petrov) — read Part I (storage engines — B-Trees and LSM-Trees in real implementation depth) if you want to go beyond Chapter 5's interview-level treatment; Part II (distributed systems specifics) overlaps significantly with DDIA's Ch.5–9 and can be skipped unless you want a second, more implementation-focused perspective on the same material.

5. A "Fundamentals of Distributed Systems" course/text (several exist — Educative and MIT's freely available 6.824 course notes/lectures are both reasonable) — genuinely optional, for engineers who want research-level grounding beyond what any interview will ever probe. Given your 9 years of practical experience and clear goal (interview-readiness at Tier-1/Tier-2 companies, not distributed-systems research), this is explicitly the first thing to cut if time is limited — it's real depth, but it's the least interview-ROI item on this entire list.

24.E Courses

Platform	Cost model (as of this research, verify current pricing)	Best for
ByteByteGo (the paid course/platform, distinct from the free YouTube channel)	Paid, subscription/lifetime tiers, frequent discount promotions	Interview-prep-specific, visual, tightly scoped to what actually gets asked — best if you want a single, curated paid resource and prefer visual/animated explanations over dense text
Educative — "Grokking the System Design Interview" / "Grokking Modern System Design"	Paid, subscription (~\$15–20/month range with regular discounts, per current research)	Text-based, interactive, well-structured for methodical readers; "Grokking Modern System Design" is the more current/updated version as of this research — prefer it over the older original if choosing one
YouTube (ByteByteGo, Gaurav Sen channels, Section 24.A)	Free	The right starting point for almost everyone, including you — there's genuinely no need to pay before exhausting the free tier's depth, especially combined with this roadmap's own written material
freeCodeCamp long-form courses	Free	A free alternative to a paid structured course if you want one long, guided sitting
Udemy system design courses	Paid, one-time (frequently deeply discounted)	Hit-or-miss quality — vet the specific instructor's reviews carefully before buying; not specifically recommended over the options above given the free/higher-quality alternatives that exist

Recommendation for your specific situation: given your strong practical background and the free resources' genuine quality, **start entirely free** (ByteByteGo YouTube + Gaurav Sen + this roadmap + the System Design Primer). Only consider a paid platform (ByteByteGo's paid tier or Grokking Modern System Design) in the final 3–4 weeks before real interviews specifically for its **mock-interview and question-bank breadth** —

Chapter 28 of this roadmap gives you 20 mocks, which is a strong base, but a paid platform's larger, community-validated question bank is a reasonable investment if budget allows once you're past the learning phase and into pure practice-volume mode.

Next → [Chapter 25: Study Plans](#) — the 16-week plan, daily routines (2hr/3hr/4hr), and the spaced-revision schedule.